

Урок 30. Корутины. Практика. Различные сценарии

25 декабря 2020

В этом уроке рассматриваем различные сценарии использования Flow и suspend, которые могут пригодиться в работе

Валидация текста. Flow + combine

Сценарий

На экране есть два поля ввода: Имя и Возраст. По мере ввода текста в эти поля мы хотим выполнять валидацию введенных данных и получать результат в виде Boolean. Этот результат может быть использован для отображения (или enabled статуса) кнопки Submit, например. Т.е. кнопка будет видна (активна) только если введенные данные валидны.

Реализация

Для каждого текстового поля создаем в модели отдельный StateFlow и метод для передачи данных в этот Flow:

```
1 private val _name = MutableStateFlow("")
2 private val _age = MutableStateFlow("")
3
4 fun setName(name: String) {
5     _name.value = name
6 }
7
8 fun setAge(age: String) {
9     _age.value = age
10 }
```

Эти методы вызываем в Activity (фрагменте) при изменении текста в полях:

```
1 editTextName.addTextChangedListener { model.setName(it.toString()) }
2 editTextAge.addTextChangedListener { model.setAge(it.toString()) }
```

В результате в модели у нас есть два Flow, которые транслируют тексты из полей ввода. Используем combine, чтобы объединить их:

```
1 val dataIsValid: LiveData<Boolean> = combine(_name, _age) { name, age ->
2     isNameValid(name) && isAgeValid(age)
3 }.asLiveData()
```

При поступлении новых значений в _name или _age мы в combine будем получать последние значения этих Flow. Выполняем проверки на валидность и возвращаем, как Boolean. В итоге combine предоставляет Flow<Boolean>, который при любых изменениях текста в полях ввода возвращает нам результат проверки валидности.

Добавляем asLiveData и получаем на выходе LiveData<Boolean>, которую можно использовать в биндинге для изменения видимости (активности) кнопки.

Фильтр данных. Flow + combine + mapLatest (flatMapLatest)

Сценарий

Сценарий похож на предыдущий. Только вместо валидации введенного текста мы используем его как фильтр для получения данных из БД (или с сервера).

Реализация

В модели снова создаем два StateFlow и методы для передачи данных в них:

```
1 private val _name = MutableStateFlow("")
2 private val _age = MutableStateFlow("")
3
4 fun setName(name: String) {
5     _name.value = name
6 }
7
8 fun setAge(age: String) {
9     _age.value = age
10 }
```

Вызываем эти методы при изменении текстов:

```
1 | editTextName.addTextChangedListener { model.setName(it.toString()) }
2 | editTextAge.addTextChangedListener { model.setAge(it.toString()) }
```

В итоге у нас в модели есть два Flow, которые транслируют изменения текста в полях ввода. Мы хотим эти значения использовать как фильтр в методе запроса данных. Это может быть метод UseCase или репозитория:

```
1 | suspend fun fetchData(filter: Filter): Data
```

Т.е. нам надо будет данные из двух Flow упаковывать в объект Filter(name, age) и вызывать метод fetchData:

```
1 | val filteredData: LiveData<Data> = combine(_name, _age) { name, age ->
2 |     Filter(name, age)
3 | }.mapLatest { filter ->
4 |     fetchData(filter)
5 | }.asLiveData()
```

В combine пакуем данные из обоих Flow в объект Filter. Оператор mapLatest будет отменять текущий запрос fetchData, если пришел новый объект Filter (когда поменялся текст в одном из Flow). А asLiveData конвертирует Flow в LiveData, которую можно использовать в биндинге для отображения полученных данных.

Если метод запроса данных (fetchData) возвращает не просто Data, а Flow<Data>:

```
1 | fun fetchData(filter: Filter): Flow<Data>
```

то вместо mapLatest необходимо использовать flatMapLatest:

```
1 | val filteredData: LiveData<Data> = combine(_name, _age) { name, age ->
2 |     Filter(name, age)
3 | }.flatMapLatest { filter ->
4 |     fetchData(filter)
5 | }.asLiveData()
```

Данные сразу из кэша, затем с сервера. Flow

Сценарий

Мы хотим сделать так, чтобы при запросе мы сразу получали данные из кэша, а потом уже свежие данные с сервера.

Реализация

Используем билдер flow и в нем реализуем всю логику:

```
1 | fun fetchData(id: Int): Flow<Data> = flow {
2 |     val cachedData = cache.getData(id)
3 |     if (cachedData != null) {
4 |         emit(cachedData)
5 |     }
6 |
7 |     val apiData = apiService.fetchData(id)
8 |     cache.putData(id, apiData)
9 |     emit(apiData)
10 | }
```

Сначала проверяем кэш. Если данные есть, то отправляем их получателю (emit). Далее получаем данные от сервера suspend методом Retrofit, обновляем кэш и отправляем их получателю.

В этот сценарий можно добавить поддержку State паттерна и если в кэше нет данных то вместо них слать State.Loading, чтобы экран показал индикатор загрузки, пока идет запрос к серверу.

Периодическая загрузка данных. Flow.

Сценарий

Необходимо создать Flow, который с определенным интервалом будет получать данные с сервера и отправлять их нам.

Реализация

Используем билдер flow:

```

1 fun fetchDataWithPeriod(): Flow<Data> = flow {
2     while(true) {
3         val data = apiService.fetchData()
4         emit(data)
5         delay(10_000)
6     }
7 }

```

Внутри делаем бесконечный цикл с паузой. Когда корутина получателя будет отменена, то цикл прервется.

Если нужна обработка ошибок, то не забывайте про Flow операторы `catch`, `retry`, `retryWhen`. Либо просто оборачивайте вызов `apiService` в `try-catch`.

Если надо получить Flow, который будет работать на несколько получателей, то используйте `shareIn` или `stateIn`.

Разовый запрос или обновляемые данные. Flow + emitAll.

Сценарий

На экране мы отображаем данные из БД. В настройках нашего приложения есть чекбокс `refreshIsEnabled`. Этим чекбоксом мы даем пользователю возможность выбора, в каком режиме отображать данные.

Если чекбокс выключен, то мы один раз запрашиваем данные из БД и отображаем их. Все последующие изменения данных в БД никак не повлияют на содержимое экрана. Т.е. в какой-то момент данные на экране станут не актуальными.

Если чекбокс включен, то нам надо, чтобы данные на экране всегда были актуальными. Т.е. обновлялись при их изменении в БД.

Реализация

Добавляем два способа получить данные из БД с помощью Room:

```

1 @Query("SELECT * FROM data")
2 suspend fun getAll(): List<Data>
3
4 @Query("SELECT * FROM data")
5 fun getAllFlow(): Flow<List<Data>>

```

Первый - это однократное получение данных. Второй - это Flow, который будет отправлять нам данные при каждом их изменении.

Два этих способа можно объединить в один Flow. А он уже под капотом пусть сам читает префы и разбирается, какой именно способ надо использовать.

Код:

```

1 fun getData(): Flow<Data> = flow {
2     if (refreshIsEnabled) {
3         emit(dataDao.getAll())
4     } else {
5         emitAll(dataDao.getAllFlow())
6     }
7 }

```

Создаем Flow. В зависимости от чекбокса `refreshIsEnabled`, мы либо отправляем (`emit`) данные, полученные из БД однократно, либо создаем перенаправление (`emitAll`), которое будет отправлять нам актуальные данные из БД.

В итоге мы можем использовать один Flow и UI ничего не будет знать об этой логике.

Базовые классы для UseCase

В гугловском приложении `iosched` активно используются `UseCase`, которые либо имеют `suspend` функцию, либо возвращают Flow. У каждого из этих типов есть базовый класс, куда вынесена работа с диспетчером и обработка ошибок.

Я здесь приведу код этих классов целиком, потому что ссылка на них может со временем устареть. Возможно, вы найдете этот код полезным при создании своих `UseCase` для работы с корутинами.

suspend UseCase

```

1  abstract class UseCase<in P, R>(private val coroutineDispatcher: CoroutineDispatcher) {
2
3      suspend operator fun invoke(parameters: P): Result<R> {
4          return try {
5              withContext(coroutineDispatcher) {
6                  execute(parameters).let {
7                      Result.Success(it)
8                  }
9              }
10         } catch (e: Exception) {
11             Timber.d(e)
12             Result.Error(e)
13         }
14     }
15
16     @Throws(RuntimeException::class)
17     protected abstract suspend fun execute(parameters: P): R
18 }

```

[источник](#)

Используется withContext чтобы сменить поток, если необходимо. А также чтобы ошибки возможных вызываемых в методе execute корутин не пошли вверх в родительскую корутину. withContext перехватит эти ошибки и выбросит в код, где они будут пойманы в try-catch. Потому что под капотом withContext работает coroutineScope.

Классу наследнику остается только реализовать suspend метод execute.

Flow UseCase

```

1  abstract class FlowUseCase<in P, R>(private val coroutineDispatcher: CoroutineDispatcher) {
2      operator fun invoke(parameters: P): Flow<Result<R>> = execute(parameters)
3          .catch { e -> emit(Result.Error(Exception(e))) }
4          .flowOn(coroutineDispatcher)
5
6      protected abstract fun execute(parameters: P): Flow<Result<R>>
7  }

```

[источник](#)

Оператор catch поймает ошибку и отправит ее получателю в упаковке Result.Error. Оператор flowOn используется для смены потока.

Классу наследнику остается только реализовать метод execute, который возвращает Flow.

Также рекомендую посмотреть [статью](#) из серии Android Mad Skills. В ней в качестве примера создаются suspend функция и Flow для работы с локацией.